

The Dot Product, Worked Out by Hand

The single most important operation in retrieval — multiply component-wise and add — shown geometrically and numerically, including exactly when it agrees with cosine and when its sensitivity to length helps or hurts.

What this document does. It computes the dot product of a query embedding against three documents by hand, decomposes it into $\|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$ to expose its two ingredients (alignment and length), and shows the consequence: dot product rewards *long* vectors, so it ranks the query’s “louder twin” above an otherwise closer document. Numbers from Appendix A; the vectors match the cosine, Euclidean, and normalisation PDFs.

1 The formula

For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ the dot (inner) product is

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

Two readings of the *same* number. The left is mechanical: multiply matching components, add them up. The right is geometric: it equals the two lengths times the cosine of the angle between them. Retrieval lives on this identity — the dot product secretly contains both “how aligned” and “how big.”

Intuition

Picture shining a light straight down onto $\mathbf{v}$ from the direction of $\mathbf{u}$: the dot product is the length of $\mathbf{v}$ ’s shadow along $\mathbf{u}$, scaled by $\|\mathbf{u}\|$. A long vector pointing the right way casts a long shadow — a big dot product. So the dot product blends two distinct things into one score: *agreement of direction* and *sheer magnitude*. Whether mixing them is good or bad depends entirely on whether your embeddings are normalised.

2 Worked by hand

Query $\mathbf{q} = (2, 1, 2)$ against $\mathbf{a} = (1, 0, 2)$, $\mathbf{b} = (0, 3, 1)$, and $\mathbf{c} = (4, 2, 4) = 2\mathbf{q}$:

$$\mathbf{q} \cdot \mathbf{a} = 2 \cdot 1 + 1 \cdot 0 + 2 \cdot 2 = 6,$$

$$\mathbf{q} \cdot \mathbf{b} = 2 \cdot 0 + 1 \cdot 3 + 2 \cdot 1 = 5,$$

$$\mathbf{q} \cdot \mathbf{c} = 2 \cdot 4 + 1 \cdot 2 + 2 \cdot 4 = 18.$$

Ranking by dot product: $\mathbf{c}(18) > \mathbf{a}(6) > \mathbf{b}(5)$.

Decompose each into alignment $\times$ length. Using $\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$ with $\|\mathbf{q}\| = 3$:

	$\ \mathbf{v}\ $	$\cos \theta$	$\ \mathbf{q}\ \ \mathbf{v}\ \cos \theta$	dot
a	2.2361	0.8944	$3 \times 2.2361 \times 0.8944$	6
b	3.1623	0.5270	$3 \times 3.1623 \times 0.5270$	5
c	6.0000	1.0000	$3 \times 6.0000 \times 1.0000$	18

c wins not because it is better aligned than **a** (both are well-aligned; **c** is perfectly aligned) but mostly because it is *long*.

Mini-example · doubling a vector doubles its score

$\mathbf{c} = 2\mathbf{q}$, so $\mathbf{q} \cdot \mathbf{c} = \mathbf{q} \cdot 2\mathbf{q} = 2(\mathbf{q} \cdot \mathbf{q}) = 2\|\mathbf{q}\|^2 = 2 \cdot 9 = 18$. Compare $\mathbf{q} \cdot \mathbf{q} = 9$: the same content at “twice the volume” earns exactly double the dot-product score, even though its *meaning* (direction) is unchanged. Cosine (PDF #5) gave both the same score of 1. This single example is the whole difference between the two metrics.

3 When length-sensitivity helps — and when it hurts

Hurts (uncontrolled). If document vectors vary in length for reasons unrelated to relevance — longer passages, more tokens pooled — raw dot product will systematically over-rank the long ones. Document **c** beating **a** here is exactly that artefact. The standard fix is to *normalise* (PDF #8), after which dot product equals cosine and length stops leaking in.

Helps (deliberate). Some models bake useful signal *into* the norm — e.g. “how confident / how much information is in this vector.” MIPS (maximum-inner-product search) is then the right query, because you *want* a long, confident match to outrank a short, hesitant one. The lesson: dot product is the right tool precisely when magnitude is meaningful, and the wrong one when it is noise.

Intuition

Rule of thumb. **Normalised embeddings $\Rightarrow$ dot product and cosine are identical**, so use dot product (it is one multiply-add per dimension, no square roots). **Un-normalised embeddings $\Rightarrow$ decide first** whether length carries meaning; if not, normalise before you ever compare. Most modern embedding APIs already return unit vectors, which is why production retrieval is overwhelmingly dot-product search underneath.

Equation cheat-sheet

$$\mathbf{u} \cdot \mathbf{v} = \sum_i u_i v_i = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta, \quad \mathbf{v} \cdot \mathbf{v} = \|\mathbf{v}\|^2$$

$$\mathbf{u} \cdot (\alpha \mathbf{v}) = \alpha (\mathbf{u} \cdot \mathbf{v}), \quad \text{unit vectors} \Rightarrow \mathbf{u} \cdot \mathbf{v} = \cos(\mathbf{u}, \mathbf{v})$$

Appendix A · Reference implementation

```
import numpy as np

q = np.array([2,1,2])
for name, v in [("a",[1,0,2]), ("b",[0,3,1]), ("c",[4,2,4])]:
    print(name, q @ np.array(v))
# a 6   b 5   c 18   ->   dot ranks c > a > b   (c = 2q, length wins)
```

Internal learning material. Numbers reproducible from the appendix code. v1.0